

Project Proposal Form

Institute of Information Technology (IIT)

University of Dhaka

Student's Name: Farhan Tausif

Student's Roll: 1440

Phone: 01305-584050

DeHalu: Agentic Hallucination Detection and Mitigation for local CodeLLMs

Motivation

Nowadays, students use local code-generating LLMs (CodeLLMs) because State-Of-The-Art (SOTA) models are expensive and unaffordable for a student-budget, whereas local alternatives provide completely free, unlimited assistance without any subscription fees. However, these models hallucinate and produce incorrect or misleading code. Since the CodeLLMs sound very confident, users may trust and integrate the hallucinated code that fails to compile, misuses APIs, breaks dependencies, or does not meet the actual requirements. In some cases, this can even introduce critical vulnerabilities into a project.

Project Description

This project aims to build an agentic tool that acts as a verification gateway for local CodeLLMs, focused on detecting and mitigating hallucinations in generated code before the output is trusted by the user, is called DeHalu. It accepts a user's natural-language programming prompt, generates code with a local CodeLLM, and then passes the generated output through a structured, execution-free hallucination-detection and mitigation pipeline.

DeHalu will not rely only on model confidence. Instead of returning raw model output directly to the user, DeHalu provides a verified or repaired result with quantitative metrics and evidence explaining what was checked and why the final decision was made.

Proposed Solution

To achieve reliable code generation, DeHalu implements a multi-stage, agentic verification pipeline by combining automated inference, polyglot static analysis, quantitative model metrics, and strict LLM-as-a-Judge evaluation. The system iteratively detects and mitigates hallucinations without the security risks or overhead of dynamic sandbox execution.

The core workflow of the proposed solution is as follows:

- **Prompt Intake and Automated Inference (Phase 0):** The user provides only a natural-language prompt. The tool automatically infers the target language, framework, runtime, and constraints, ensuring a fully automated, zero-friction pipeline.
- **Preventative Mitigation via Chain-of-Thought [1]:** Before code generation, the tool

forces the local CodeLLM to use structured reasoning prompts to decompose the task, plan the structural design, and guide the logic, heavily reducing functional requirement deviations.

- **Code Generation & Metric Logging (Phase 1):** A local CodeLLM (e.g., 3B-7B parameter model optimized for 16GB RAM hardware) generates the initial implementation. During generation, the tool actively logs Token Log-Probabilities and Predictive Entropy to capture the model's internal uncertainty.
- **Claim Extractions:** The generated code is decomposed into checkable claims, that may include imported packages, referenced APIs, etc.
- **Execution-Free Static Analysis (Phase 2):** The code is inspected strictly without execution using AST parsers (e.g., Tree-sitter) and SAST tools (e.g., Semgrep) to identify structural flaws, syntax violations, and code quality hallucinations.
- **Tool-Backed Validation:** DeHalu uses symbol indexers to validate that referenced packages and APIs actually exist in the target language's environment and are used correctly.
- **Quantitative Metric Engine:** Utilizing static analysis results, the system calculates the Micro Hallucination Number (MiHN) and Macro Hallucination Rate (MaHR) to quantify API hallucinations, and TR-S for structural repetition. It mathematically proves hallucinations by correlating static invalid references with internal high-entropy/log-prob spikes from the generation phase.
- **LLM-as-a-Judge [4] Verification (Phase 3):** A single, highly capable judge role evaluates the code for functional correctness and requirement deviations. To ensure reliability without execution, the judge operates on a highly structured grading rubric with explicit deduction criteria, similar to establishing a strict "নম্বর বন্টন" (mark distribution) for ICT assessments.
- **Chain-of-Verification [5] (CoVe) for Mitigation:** DeHalu converts generated claims into targeted verification questions and checks them against ground-truth facts from the static analysis tools.
- **Policy Decision:** All verification signals (static errors, metric proofs, and judge rubric scores) feed into a policy decision layer that makes a deterministic judgment: accept the code or trigger repair.
- **Iterative Mitigation and Repair (Phase 4):** The tool focuses on parameter-free mitigation methods [6]. If mitigation is triggered, the same local CodeLLM switches to a Repair Agent role via a strict system prompt. It receives the failed code and the fact-checked CoVe evidence, fixes the issues without guessing, and sends the repaired code back through the verification pipeline.

References:

- [1] Y. Lee et al., "Hallucination by Code Generation LLMs: Taxonomy, Benchmarks, Mitigation, and Challenges," arXiv preprint arXiv:2504.20799, 2025.
- [2] J. Cheng et al., "Chain-of-Thought Prompting Obscures Hallucination Cues in Large Language Models: An Empirical Evaluation," arXiv preprint arXiv:2506.17088, 2025
- [3] B. Yang et al., "Hallucination Detection in Large Language Models with Metamorphic Relations," Proceedings of the ACM on Software Engineering (PACMSE), Vol. 2, No. FSE, Article FSE020, 2025
- [4] J. He et al., "LLM-as-a-Judge for Software Engineering: Literature Review, Vision, and the Road Ahead," ACM Transactions on Software Engineering and Methodology, vol. 34, no. 5,

2026.

[5] S. Dhuliawala et al., "Chain-of-Verification Reduces Hallucination in Large Language Models," Findings of the Association for Computational Linguistics: ACL 2024, pp. 3563-3578, 2024.

[6] C. Gao et al., "A Systematic Literature Review of Code Hallucinations in LLMs: Characterization, Mitigation Methods, Challenges, and Future Directions for Reliable AI," arXiv preprint arXiv:2511.00776, 2025.

Supervisor's Name: Dr. Kazi Muheymin-Us-Sakib

Signature of the supervisor: _____

Date: _____

Proposal Presentation Feedback:

Static analysis only

Which error raises by hallucination

Why typical solutions are not working on hallucination error

Saeed Siddik

Assistant Professor,

IIT, DU

19/05/26

Midterm Presentation Feedback: